Guia de estudo — as 29 rubricas no projeto Pedro

Para que serve: aprender cada rubrica em poucos minutos. Cada ficha responde quatro perguntas: o que é o conceito, em que consiste no Pedro, onde está no projeto e como ver funcionando.

Como usar: leia na ordem das fases. Cada fase constrói sobre a anterior. Para o detalhe e as evidências de cada rubrica, abra docs/respostas_rubricas.md na seção com o mesmo código.

1. O projeto em um minuto

O Pedro é um assistente de compras de peças de eletrodomésticos. Todos os dados são fictícios. O usuário pede uma peça e uma quantidade; o Pedro compara os fornecedores e recomenda um.

- **O núcleo é comparar fornecedores.** A função comparar em src/pedro/business.py descarta ofertas por estoque, pedido mínimo, prazo ou orçamento e ordena as que sobram.
- **A ideia que organiza tudo:** o modelo decide o que responder; os dados quem coloca é o sistema. Preços, totais e citações nunca vêm da "memória" do modelo.
- **Nada é publicado sem verificação.** Cada resposta passa por uma checagem exata e por um modelo de NLI antes de aparecer.
- **Todo dinheiro é inteiro, em centavos.** Nunca float.

2. Mapa do projeto

Pasta ou arquivo	O que tem
app.py	A interface Streamlit (só o chat)
src/pedro/	Toda a lógica reutilizável
scripts/demonstrar_*.py	Demonstrações com o modelo real; salvam um JSON em docs/rastros/
notebooks/	Cinco notebooks didáticos, um por fase, com duplos de teste
tests/	156 testes automáticos
evals/	Casos de avaliação e o avaliador do LangSmith
knowledge/	Corpus fictício de políticas e o índice FAISS salvo
datasets/	Catálogo SQLite sintético: 18 categorias, 4.320 peças, 12.960 ofertas
docs/	Respostas às rubricas, relatório, matriz, evidências e rastros

3. As cinco fases

Fase	Rubricas	Ideia	Notebook
1	R01–R06	Uma cadeia que extrai a solicitação, e a arquitetura do agente	01_chain_mvp
2	R07–R12	A cadeia vira um agente com ferramentas (ReAct)	02_react_ferramentas
3	R13–R18	O agente lembra: memória e conversa	03_memoria_chatbot
4	R19–R23	O agente consulta dados: SQL, APIs e RAG	04_rag_text_to_query
5	R24–R29	Tudo junto, verificado, na interface e avaliado	05_integrador_avaliacao

4. Fase 1 — Chain executável e arquitetura

R01 — Anatomia, tipo de agente e maturidade

- **O conceito:** um agente tem partes: motor cognitivo (o LLM), ferramentas, memória, base de conhecimento e orquestrador. Pode ser reativo (regras fixas), deliberativo (raciocina) ou híbrido.
- **No Pedro:** é um agente híbrido. A camada reativa são regras que resolvem o que é simples sem chamar o modelo. A camada deliberativa é o LLM, que escolhe ferramentas.
- **Onde está:** src/pedro/anatomia.py (descrever_anatomia, inventario_langchain); as duas camadas vivem em src/pedro/agent.py (criar_fluxo).
- **Ver funcionando:** notebook 01, seções R01; tests/test_anatomia.py.
- **Para lembrar:** híbrido = regras para o previsível, modelo para o resto.

R02 — Planejamento e memória aumentada

- **O conceito:** planejar é dividir um pedido complexo em passos. Memória aumentada é guardar fora do modelo o que ele precisa lembrar.
- **No Pedro:** quando uma mensagem tem duas ou mais intenções, o modelo escreve um plano validado (PlanoTurno), com um pedido por intenção. Cada pedido é resolvido pelo caminho normal, com a sua verificação.
- **Onde está:** src/pedro/planner.py (intencoes_detectadas, planejar_turno, validar_plano); memória em src/pedro/memory.py.
- **Ver funcionando:** scripts/demonstrar_r02.py; rastro docs/rastros/r02_planejamento_llm.json.
- **Para lembrar:** o plano não pode conter dados que o usuário não escreveu.

R03 — Modelos de prompt reutilizáveis

- **O conceito:** PromptTemplate e ChatPromptTemplate são prompts com variáveis, escritos uma vez e usados muitas vezes.
- **No Pedro:** dez modelos com nome, por exemplo MODELO_EXTRACAO e MODELO_AGENTE.
- **Onde está:** src/pedro/prompts.py.
- **Ver funcionando:** notebook 01, seção R03; tests/test_prompts.py.
- **Para lembrar:** o prompt é código: tem nome, variáveis e teste.

R04 — Output parser com saída JSON

- **O conceito:** um parser transforma o texto do modelo em dados estruturados e valida o formato.
- **No Pedro:** PydanticOutputParser com os contratos de contracts.py. Há uma checagem extra de JSON completo, porque um parser tolerante pode aceitar um JSON cortado e perder dados em silêncio.
- **Onde está:** src/pedro/chains.py (_json_completo, criar_lcel_robusta); contratos em src/pedro/contracts.py (Extracao, Solicitacao).
- **Ver funcionando:** notebook 01, seções R04; scripts/demonstrar_fase1.py; tests/test_saida_estruturada.py.
- **Para lembrar:** validar a forma não basta; é preciso validar que a saída está inteira.

R05 — invoke, batch, stream e @chain

- **O conceito:** todo Runnable tem três formas de execução: invoke (uma entrada), batch (várias em paralelo) e stream (pedaços enquanto chegam). @chain transforma uma função Python em Runnable.
- **No Pedro:** a mesma cadeia de extração é executada das três formas e escrita também com @chain.
- **Onde está:** src/pedro/chains.py (criar_chain, criar_chain_robusta, criar_lcel_parcial).
- **Ver funcionando:** notebook 01, seções R05; scripts/demonstrar_fase1.py (mede os tempos).
- **Para lembrar:** batch foi cerca de três vezes mais rápido que três invoke em série.

R06 — LCEL: declarativo contra imperativo

- **O conceito:** LCEL compõe passos com o operador | (declarativo). A alternativa é escrever o fluxo como código normal (imperativo).
- **No Pedro:** a mesma cadeia com reparo existe nos dois estilos, e um laboratório compara o que cada um deixa ver no rastro.
- **Onde está:** src/pedro/chains.py (criar_lcel, criar_lcel_robusta, criar_chain_robusta) e src/pedro/laboratorio.py (comparar_estilos, ensaiar).
- **Ver funcionando:** notebook 01, seção R06.
- **Para lembrar:** LCEL mostra cada passo no rastro; o imperativo é mais fácil de ler quando há ramificações.

5. Fase 2 — Agente com ferramentas e ReAct

R07 — Comparar orquestradores

- **O conceito:** o orquestrador é o que coordena modelo, ferramentas e memória. LangChain clássico e LangGraph são duas opções.
- **No Pedro:** o mesmo agente de duas ferramentas foi construído nos dois e comparado. O produto usa LangChain.
- **Onde está:** src/pedro/orquestradores.py (agente_langchain, agente_langgraph); texto em docs/comparacoes.md.
- **Ver funcionando:** scripts/demonstrar_r07.py; rastro docs/rastros/r07_orquestradores_llm.json.
- **Para lembrar:** a escolha se justifica com uma comparação executada, não com opinião.

R08 — ReAct com initialize_agent

- **O conceito:** ReAct é o ciclo Thought, Action, Observation: o modelo pensa, usa uma ferramenta, lê o resultado e repete.
- **No Pedro:** um agente ReAct clássico com initialize_agent, duas ferramentas e memória de conversa.
- **Onde está:** src/pedro/legacy.py (criar_react_textual).
- **Ver funcionando:** notebook 02, seção R08; scripts/demonstrar_r08.py; tests/test_legacy.py.
- **Para lembrar:** com a memória no prompt, o agente entende "e dessa peça?"; sem ela, pede o ID de novo.

R09 — Chains sequenciais, paralelas e RouterChain

- **O conceito:** cadeias podem rodar em sequência, em paralelo (RunnableParallel) ou ser escolhidas por um roteador.
- **No Pedro:** uma composição paralela e dois roteadores: um por regras (RouterCompras) e um em que o LLM escolhe o destino.
- **Onde está:** src/pedro/chains.py (composicao_paralela) e src/pedro/legacy.py (RouterCompras, criar_router_llm, executar_rota).
- **Ver funcionando:** notebook 02, seção R09; scripts/demonstrar_r09.py.
- **Para lembrar:** nas mesmas perguntas, as regras acertaram 2 de 7 rotas e o LLM 6 de 7.

R10 — API externa por function calling

- **O conceito:** function calling é o modelo pedir a execução de uma função com argumentos estruturados.
- **No Pedro:** a API pública ViaCEP é a ferramenta consultar_cep. A cidade publicada vem da resposta da API, não do modelo.
- **Onde está:** src/pedro/integrations.py (consultar_cep); ferramenta em src/pedro/agent.py; ancoragem em src/pedro/verification.py (ancorar_cep).
- **Ver funcionando:** scripts/demonstrar_r10.py; tests/test_policy_cep.py.
- **Para lembrar:** o modelo escolhe chamar; o dado publicado é o que a API devolveu.

R11 — PTCF, few-shot e chain-of-thought

- **O conceito:** PTCF é estruturar o prompt em Persona, Tarefa, Contexto e Formato. Few-shot é dar exemplos. Chain-of-thought é pedir o raciocínio antes da resposta.
- **No Pedro:** três variantes do prompt de extração foram medidas nas mesmas dez solicitações.
- **Onde está:** src/pedro/prompts.py (MODELO_EXTRACAO, MODELO_EXTRACAO_FEWSHOT, MODELO_EXTRACAO_COT, MODELO_AGENTE); src/pedro/chains.py (criar_variantes_de_prompt).
- **Ver funcionando:** scripts/demonstrar_r11.py; rastro docs/rastros/r11_prompts_llm.json.
- **Para lembrar:** zero-shot acertou 6 de 10 solicitações; few-shot e few-shot com CoT, 10 de 10.

R12 — Componentes de confiabilidade

- **O conceito:** parsers com tratamento de erro, fallbacks, memória, índice vetorial e callbacks para saber o que aconteceu.
- **No Pedro:** um laboratório de saídas quebradas mostra qual camada resolve cada defeito. O callback Rastro registra ferramentas, verificações e erros de cada turno.
- **Onde está:** src/pedro/laboratorio.py (SAIDAS_QUEBRADAS, ensaiar); src/pedro/agent.py (classe Rastro).
- **Ver funcionando:** notebook 01, seção R04; notebook 02, seção R12.
- **Para lembrar:** sem registrar o motivo de cada rejeição, não dá para corrigir o agente.

6. Fase 3 — Chatbot com memória

R13 — Memória de curto e longo prazo (CoALA)

- **O conceito:** a taxonomia CoALA separa memória de trabalho (curto prazo), episódica (o que aconteceu), semântica (fatos) e procedimental (como agir).
- **No Pedro:** mensagens e rascunho de compra (trabalho), episódios com embeddings (episódica), preferências e políticas (semântica), prompts e regras (procedimental).
- **Onde está:** src/pedro/memory.py (mapa_memorias, Historico, salvar_rascunho, salvar_preferencia, salvar_episodio).
- **Ver funcionando:** notebook 03, seção R13; scripts/demonstrar_fase3.py.
- **Para lembrar:** cada tipo de memória responde uma pergunta diferente: o que estamos fazendo, o que aconteceu, o que é verdade, como agir.

R14 — Janela de contexto e busca vetorial

- **O conceito:** o modelo só vê uma janela das últimas mensagens. O que ficou fora se recupera por similaridade de embeddings.
- **No Pedro:** janela de 12 mensagens e recuperação dos três episódios mais parecidos, com FastEmbed.
- **Onde está:** src/pedro/memory.py (JANELA_MENSAGENS, recuperar, FastEmbedder).
- **Ver funcionando:** notebook 03; rastro docs/rastros/fase3_memoria_llm.json.
- **Para lembrar:** janela para o recente, vetor para o antigo.

R15 — Comparar ferramentas de memória

- **O conceito:** existem bibliotecas especializadas em memória de agentes, como LangMem, Mem0 e Letta.
- **No Pedro:** a memória própria foi comparada com o LangMem em execução real; Mem0 e Letta ficam como comparação de arquitetura.
- **Onde está:** src/pedro/orquestradores.py (agente_com_langmem); texto em docs/comparacoes.md.
- **Ver funcionando:** scripts/demonstrar_r15.py; rastro docs/rastros/r15_memoria_llm.json.
- **Para lembrar:** no Pedro o sistema decide o que salvar; no LangMem quem decide é o modelo.

R16 — ConversationBufferMemory

- **O conceito:** a memória mais simples: guardar toda a conversa e colocá-la inteira no prompt.
- **No Pedro:** um chatbot clássico com ConversationChain, para mostrar que o buffer cresce a cada turno.

- **Onde está:** src/pedro/legacy.py (criar_chatbot_buffer, MODELO_CHATBOT).
- **Ver funcionando:** notebook 03, seção R16.
- **Para lembrar:** o buffer cresce sem limite; por isso o produto usa janela e busca vetorial.

R17 — Intenção e extração de entidades

- **O conceito:** entender o que o usuário quer (intenção) e os dados que ele deu (entidades: peça, quantidade, orçamento, prazo).
- **No Pedro:** regras para o ID da peça e um intérprete few-shot (MudancaSolicitacao) que decide se a mensagem é uma compra e o que ela muda.
- **Onde está:** src/pedro/dialogue.py (interpretar_compra, interpretar_turno, criar_interprete).
- **Ver funcionando:** scripts/demonstrar_r17.py; tests/test_dialogue.py.
- **Para lembrar:** só regras acertaram 8 de 14 mensagens; regras mais modelo, 14 de 14.

R18 — Contexto e decomposição em conversa real

- **O conceito:** manter o estado ao longo de vários turnos e dividir mensagens com vários pedidos.
- **No Pedro:** uma conversa real de oito turnos muda quantidade, tira o prazo, salva uma preferência, pergunta um CEP e uma política, e volta à compra sem perder o estado.
- **Onde está:** src/pedro/dialogue.py (rascunho por sessão) e src/pedro/planner.py; fluxo em src/pedro/agent.py.
- **Ver funcionando:** scripts/demonstrar_fase3.py; rastro docs/rastros/fase3_memoria_llm.json.
- **Para lembrar:** o Pedro só pergunta o dado que bloqueia (a quantidade); orçamento e prazo são filtros opcionais.

7. Fase 4 — SQL, APIs e RAG

R19 — Ferramentas hardcoded e semânticas

- **O conceito:** uma ferramenta hardcoded executa código fixo; uma semântica depende de busca por significado.
- **No Pedro:** seis ferramentas. comparar_ofertas é cálculo fixo; consultar_politicas busca por significado no índice FAISS.
- **Onde está:** src/pedro/agent.py (função ferramentas); papéis descritos em src/pedro/anatomia.py (PAPEIS).
- **Ver funcionando:** tests/test_tools_flow.py; notebook 05.
- **Para lembrar:** contas e regras ficam em código fixo; o modelo só escolhe quando chamar.

R20 — APIs internas e externas

- **O conceito:** uma API interna é da própria aplicação; uma externa é de terceiros e pode falhar.
- **No Pedro:** interna, o catálogo SQLite em modo somente leitura; externa, o ViaCEP, com tratamento de CEP inválido, inexistente e serviço fora do ar.
- **Onde está:** src/pedro/catalog.py e src/pedro/integrations.py.
- **Ver funcionando:** scripts/demonstrar_r10.py; tests/test_rag_integrations.py.
- **Para lembrar:** toda chamada externa precisa de um plano para quando falha.

R21 — Text-to-query

- **O conceito:** transformar uma pergunta em linguagem natural em uma consulta ao banco.
- **No Pedro:** dois caminhos. Com ID de peça, um plano Pydantic vira um SELECT parametrizado. Para perguntas abertas, o LLM escreve o SQL, que passa por uma lista branca de tabelas e colunas, somente leitura, limite de linhas e uma tentativa de reparo.
- **Onde está:** src/pedro/query.py (planejar, executar, validar_sql, consultar_sql, responder_sql).
- **Ver funcionando:** notebook 04, seção R21; scripts/demonstrar_fase4.py; tests/test_text_to_sql.py.

- **Para lembrar:** o modelo escreve a consulta; quem decide se ela roda é o motor.

R22 — RAG sobre um índice FAISS existente

- **O conceito:** RAG é recuperar trechos relevantes de documentos e responder com base neles. Aqui o índice já existe em disco: é carregado, não construído.
- **No Pedro:** o índice salvo em knowledge/politicas/ só é aberto depois de conferir o manifesto (origem, modelo, dimensão, métrica e SHA-256), porque o arquivo é pickle.
- **Onde está:** src/pedro/rag.py (inspecionar_indice, carregar_indice, consultar).
- **Ver funcionando:** notebook 04, seção R22; scripts/demonstrar_r22.py; tests/test_indice_existente.py.
- **Para lembrar:** PEDRO_TRUST_POLICY_INDEX=true só depois de verificar a procedência do índice.

R23 — Falha real de recuperação, diagnosticada e corrigida

- **O conceito:** a busca vetorial pode não trazer o trecho certo. É preciso medir a falha, entender a causa e corrigir.
- **No Pedro:** a pergunta coloquial «Posso receber em várias caixas?» não traz a regra de entrega parcial no top-4. Reescrita como pergunta sobre envio parcial, a regra vai para a posição 1.
- **Onde está:** src/pedro/policy_demo.py (reescrever_consulta_entrega); src/pedro/rag.py (diagnosticar_reescrita); scripts/diagnosticar_t24.py; texto em docs/t24_recuperacao.md.
- **Ver funcionando:** notebook 04, seção T24; no chat, o comando /t24.
- **Para lembrar:** o usuário fala em "caixas", o documento fala em "entrega parcial". A reescrita aproxima os dois vocabulários.

8. Fase 5 — Agente final verificado e avaliado

R24 — Verificação factual e coerência

- **O conceito:** conferir que a resposta do modelo é fiel às fontes. NLI (inferência em linguagem natural) diz se um texto é sustentado por outro.
- **No Pedro:** duas portas. A verificação exata confere vencedor, total, fontes e dataset. O NLI confere se o texto é sustentado. Antes disso, o sistema ancora os dados: troca os números pelo resultado da ferramenta e as citações pelo texto literal do fragmento.
- **Onde está:** src/pedro/verification.py (verificar, ancorar_recomendacao, ancorar_citacoes, ancorar_cep, ancorar_catalogo) e src/pedro/nli.py (verificar_nli); texto em docs/citacoes_ancoradas.md.
- **Ver funcionando:** tests/test_core.py, tests/test_citacoes_ancoradas.py; notebook 05.
- **Para lembrar:** nenhuma recomendação contrária à regra foi publicada em nove experimentos.

R25 — Um agente com duas ou mais categorias de ferramentas

- **O conceito:** um único agente que combina SQL, RAG e API.
- **No Pedro:** o mesmo executor usa consultar_catalogo (SQL), consultar_politicas (RAG) e consultar_cep (API), além de comparar_ofertas, buscar_produtos e salvar_preferencia.
- **Onde está:** src/pedro/agent.py (ferramentas, criar_agente).
- **Ver funcionando:** scripts/demonstrar_fase3.py (as três categorias na mesma conversa).
- **Para lembrar:** uma conversa, um agente, três tipos de fonte.

R26 — create_openai_tools_agent e AgentExecutor

- **O conceito:** create_openai_tools_agent cria o agente que usa function calling; AgentExecutor roda o ciclo de chamar ferramentas até ter a resposta.
- **No Pedro:** seis ferramentas, no máximo seis iterações e uma única passada de correção quando a verificação rejeita a primeira proposta.
- **Onde está:** src/pedro/agent.py (criar_agente, criar_fluxo).

- **Ver funcionando:** tests/test_tools_flow.py; scripts/demonstrar_fase3.py.
- **Para lembrar:** os limites de iterações e de correção existem para a autonomia ser controlada.

R27 — RunnableWithMessageHistory

- **O conceito:** envolve uma cadeia e cuida de ler e gravar o histórico de cada sessão.
- **No Pedro:** o histórico fica em SQLite, por usuário e sessão. Só a resposta pública é gravada, nunca o raciocínio interno nem as propostas rejeitadas.
- **Onde está:** src/pedro/agent.py (criar_fluxo, mensagem_publica) e src/pedro/memory.py (Historico).
- **Ver funcionando:** tests/test_tools_flow.py; reabrir uma conversa na interface.
- **Para lembrar:** o que o usuário não viu não entra no histórico.

R28 — Integração no Streamlit

- **O conceito:** ligar o agente completo a uma interface.
- **No Pedro:** um chat que mostra os passos do agente, quatro cartões (fornecedor, total, entrega e origem), o conselho ou aviso de preço e a tabela de ofertas. A interface desenha a partir dos dados verificados que viajam com a mensagem, sem reler o texto.
- **Onde está:** app.py; tema em .streamlit/config.toml.
- **Ver funcionando:** ./project.sh start e abrir http://localhost:8501; scripts/demonstrar_r28.py; capturas em docs/capturas/.
- **Para lembrar:** app.py só desenha; toda a lógica fica em src/pedro/.

R29 — Avaliação com LangSmith

- **O conceito:** medir o agente com um conjunto de casos e avaliadores, tanto a qualidade da resposta quanto a trajetória (quais ferramentas usou).
- **No Pedro:** 30 casos derivados e seis avaliadores: vencedor, valor, conteúdo, trajetória, ferramenta decisiva e primeira proposta.
- **Onde está:** evals/avaliar_fluxo.py; casos em evals/casos_derivados.json; gerador em scripts/criar_casos_eval.py.
- **Ver funcionando:** rastro docs/rastros/r29_langsmith_gpt-4.1-mini_portugues.json; tests/test_evals.py.
- **Para lembrar:** vencedor 14 de 14, valor 14 de 14, conteúdo 16 de 16; trajetória completa 26 de 30. Nunca simular métricas.

9. Resumo de uma página

Rubrica	Em uma frase	Arquivo principal
R01	Agente híbrido: regras e modelo	src/pedro/anatomia.py
R02	Plano validado para mensagens com várias intenções	src/pedro/planner.py
R03	Dez modelos de prompt com nome	src/pedro/prompts.py
R04	Parser Pydantic e checagem de JSON completo	src/pedro/chains.py
R05	invoke, batch, stream e @chain	src/pedro/chains.py
R06	A mesma cadeia em LCEL e em código imperativo	src/pedro/laboratorio.py
R07	LangChain comparado com LangGraph	src/pedro/orquestradores.py
R08	Agente ReAct clássico	src/pedro/legacy.py
R09	Paralelo e roteadores	src/pedro/legacy.py
R10	ViaCEP como ferramenta	src/pedro/integrations.py
R11	PTCF, few-shot e CoT medidos	src/pedro/prompts.py
R12	Laboratório de falhas e callback Rastro	src/pedro/laboratorio.py
R13	Quatro memórias da CoALA	src/pedro/memory.py

Rubrica	Em uma frase	Arquivo principal
R14	Janela de 12 mensagens e episódios por vetor	src/pedro/memory.py
R15	Memória própria comparada com LangMem	src/pedro/orquestradores.py
R16	Chatbot com buffer completo	src/pedro/legacy.py
R17	Regras e intérprete few-shot	src/pedro/dialogue.py
R18	Oito turnos reais sem perder o estado	src/pedro/dialogue.py
R19	Ferramenta fixa e ferramenta semântica	src/pedro/agent.py
R20	SQLite interno e ViaCEP externo	src/pedro/catalog.py
R21	Plano para SELECT e text-to-SQL com lista branca	src/pedro/query.py
R22	Índice FAISS salvo, aberto só com manifesto conferido	src/pedro/rag.py
R23	Falha de recuperação medida e corrigida com reescrita	src/pedro/policy_demo.py
R24	Ancoragem, verificação exata e NLI	src/pedro/verification.py
R25	SQL, RAG e API no mesmo agente	src/pedro/agent.py
R26	Executor com seis ferramentas e uma correção	src/pedro/agent.py
R27	Histórico por sessão, só a resposta pública	src/pedro/memory.py
R28	Chat Streamlit que desenha dados verificados	app.py
R29	30 casos e seis avaliadores no LangSmith	evals/avaliar_fluxo.py

10. Roteiro de estudo sugerido

- **Dia 1 (Fases 1 e 2):** execute os notebooks 01 e 02. Depois leia as fichas R01 a R12 deste guia.
- **Dia 2 (Fases 3 e 4):** execute os notebooks 03 e 04. Leia as fichas R13 a R23.
- **Dia 3 (Fase 5):** execute o notebook 05, abra a interface e faça uma compra no chat. Leia as fichas R24 a R29.
- **Revisão:** cubra a coluna do meio do resumo e tente explicar cada rubrica em uma frase.